

Theoretical README

Lucas–Kanade Optical Flow Estimation

Contents

1	Introduction	3
2	Basic Idea of Optical Flow	3
3	Brightness Constancy Assumption	4
4	Derivation of the Optical Flow Constraint Equation	4
5	Interpretation of the Optical Flow Constraint Equation	5
6	Aperture Problem	5
7	Local Window Assumption	5
8	Least-Squares Solution	6
9	Moore–Penrose Pseudoinverse Interpretation	7
10	Rank of the Local Matrix	8
11	Image Preprocessing	8
12	Gaussian Smoothing	9
13	Spatial Image Gradients	9
14	Temporal Gradient	9
15	Construction of the Local Matrix	10
16	Why Multiple Pixels Are Used	11
17	Window Processing in the Implementation	11
18	Optical-Flow Vector	12
19	Visualization of Optical Flow	12
20	Frame-to-Frame Processing	13

21 Output Video Generation	13
22 Complete Mathematical Pipeline	14
23 Important Assumptions	15
23.1 Brightness Constancy	15
23.2 Small Motion	16
23.3 Local Constant Motion	16
23.4 Sufficient Image Texture	16
24 Advantages	16
25 Limitations	16
25.1 Large Motion	17
25.2 Brightness Changes	17
25.3 Occlusion	17
25.4 Aperture Problem	17
25.5 Window Size	17
26 Computational Interpretation	17
27 Computation Versus Visualization	18
28 Interpretation of the Final Result	18
29 Summary	18

1 Introduction

Optical flow is the apparent motion of image points between two consecutive frames of a video. Given two frames captured at times t and $t + \Delta t$, optical flow estimates how image intensity patterns move from the first frame to the second.

The Lucas–Kanade method is a classical technique for estimating optical flow. It is based on two important assumptions:

1. **Brightness constancy:** the intensity of a moving image point remains approximately constant.
2. **Local motion consistency:** pixels within a sufficiently small spatial neighborhood undergo approximately the same motion.

The present implementation estimates the horizontal and vertical motion components for local image windows and visualizes the estimated motion using arrows.

The horizontal and vertical optical-flow components are denoted by

$$u = \frac{dx}{dt}, \quad v = \frac{dy}{dt}.$$

Thus, the optical-flow vector at a point is

$$\mathbf{V} = \begin{bmatrix} u \\ v \end{bmatrix}.$$

2 Basic Idea of Optical Flow

Let the image intensity be represented by

$$I(x, y, t),$$

where

- x is the horizontal image coordinate,
- y is the vertical image coordinate,
- t is time.

Suppose a particular image point moves from

$$(x, y, t)$$

to

$$(x + dx, y + dy, t + dt).$$

Under the brightness-constancy assumption, the intensity of this moving point remains approximately unchanged:

$$I(x, y, t) = I(x + dx, y + dy, t + dt).$$

This equation is the starting point for the Lucas–Kanade derivation.

3 Brightness Constancy Assumption

The brightness constancy assumption states that a moving point retains approximately the same image intensity between two consecutive frames.

Therefore,

$$I(x, y, t) = I(x + dx, y + dy, t + dt).$$

The assumption is not exactly true in every real-world situation. It works best when illumination does not change significantly and when the observed surface has approximately constant reflectance.

4 Derivation of the Optical Flow Constraint Equation

We expand

$$I(x + dx, y + dy, t + dt)$$

using a first-order Taylor expansion around (x, y, t) :

$$I(x + dx, y + dy, t + dt) \approx I(x, y, t) + I_x dx + I_y dy + I_t dt.$$

Using brightness constancy,

$$I(x, y, t) = I(x, y, t) + I_x dx + I_y dy + I_t dt.$$

Subtracting $I(x, y, t)$ from both sides gives

$$I_x dx + I_y dy + I_t dt = 0.$$

Dividing by dt gives

$$I_x \frac{dx}{dt} + I_y \frac{dy}{dt} + I_t = 0.$$

Define

$$u = \frac{dx}{dt}, \quad v = \frac{dy}{dt}.$$

Therefore,

$$\boxed{I_x u + I_y v + I_t = 0}$$

This is known as the **Optical Flow Constraint Equation (OFCE)**.

5 Interpretation of the Optical Flow Constraint Equation

The equation

$$I_x u + I_y v + I_t = 0$$

contains two unknowns:

$$u, \quad v.$$

However, at a single pixel we have only one equation.

Therefore, one pixel is not sufficient to uniquely determine both components of the optical flow.

For example,

$$I_x u + I_y v = -I_t$$

represents a line of possible (u, v) values rather than one unique solution.

This is the fundamental ambiguity known as the **aperture problem**.

6 Aperture Problem

The aperture problem occurs because the motion of an image feature cannot generally be determined completely from the information available at a single pixel or along a simple edge.

Suppose an image contains a straight vertical edge. The intensity changes primarily in the horizontal direction, so approximately

$$I_y \approx 0.$$

The optical-flow equation becomes

$$I_x u = -I_t.$$

The vertical motion component v cannot be determined from this equation.

Therefore, additional spatial information is required.

Lucas–Kanade solves this problem by assuming that nearby pixels have approximately the same motion.

7 Local Window Assumption

Consider a small image window containing N pixels.

The Lucas–Kanade assumption is that the optical flow is approximately constant throughout this local neighborhood:

$$u_i \approx u, \quad v_i \approx v.$$

For every pixel i in the window,

$$I_{x,i}u + I_{y,i}v = -I_{t,i}.$$

For N pixels, we obtain N equations:

$$\begin{aligned} I_{x,1}u + I_{y,1}v &= -I_{t,1}, \\ I_{x,2}u + I_{y,2}v &= -I_{t,2}, \\ &\vdots \\ I_{x,N}u + I_{y,N}v &= -I_{t,N}. \end{aligned}$$

These equations can be written in matrix form as

$$A \begin{bmatrix} u \\ v \end{bmatrix} = b,$$

where

$$A = \begin{bmatrix} I_{x,1} & I_{y,1} \\ I_{x,2} & I_{y,2} \\ \vdots & \vdots \\ I_{x,N} & I_{y,N} \end{bmatrix}$$

and

$$b = - \begin{bmatrix} I_{t,1} \\ I_{t,2} \\ \vdots \\ I_{t,N} \end{bmatrix}.$$

Thus,

$$\boxed{A\mathbf{v} = b}$$

where

$$\mathbf{v} = \begin{bmatrix} u \\ v \end{bmatrix}.$$

8 Least-Squares Solution

Usually, the system

$$A\mathbf{v} = b$$

is overdetermined because there are many equations but only two unknowns. For example, for a 20×20 window,

$$N = 20 \times 20 = 400.$$

Therefore,

$$A \in \mathbb{R}^{400 \times 2}, \quad b \in \mathbb{R}^{400}.$$

Since the equations may not be satisfied exactly because of noise, approximation errors, and the assumptions of the optical-flow model, the solution is obtained by minimizing the squared error:

$$\min_{\mathbf{v}} \|\mathbf{A}\mathbf{v} - \mathbf{b}\|_2^2$$

The normal equations are

$$\mathbf{A}^T \mathbf{A} \mathbf{v} = \mathbf{A}^T \mathbf{b}.$$

If $\mathbf{A}$ has full column rank, then

$$\mathbf{A}^T \mathbf{A}$$

is invertible and

$$\mathbf{v} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

This is the classical least-squares Lucas–Kanade solution.

9 Moore–Penrose Pseudoinverse Interpretation

The least-squares solution can also be written using the Moore–Penrose pseudoinverse:

$$\mathbf{v} = \mathbf{A}^+ \mathbf{b}$$

where $\mathbf{A}^+$ denotes the Moore–Penrose pseudoinverse.
For the full-column-rank case,

$$\mathbf{A}^+ = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T.$$

Therefore,

$$\mathbf{v} = \mathbf{A}^+ \mathbf{b} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}.$$

The general pseudoinverse can be computed using the singular value decomposition

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T.$$

Then

$$\mathbf{A}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T.$$

The pseudoinverse formulation is useful because it remains well-defined even when $\mathbf{A}^T \mathbf{A}$ is singular or nearly singular.

In the implementation, the least-squares solution is computed using

$$\text{np.linalg.lstsq}(\mathbf{A}, \mathbf{b}).$$

This directly solves the least-squares problem

$$\min_{\mathbf{v}} \|\mathbf{A}\mathbf{v} - \mathbf{b}\|_2^2.$$

10 Rank of the Local Matrix

For Lucas–Kanade, the matrix A has two columns:

$$A = \begin{bmatrix} I_{x,1} & I_{y,1} \\ \vdots & \vdots \\ I_{x,N} & I_{y,N} \end{bmatrix}.$$

Therefore,

$$\text{rank}(A) \leq 2.$$

For a unique two-dimensional optical-flow estimate, the desired condition is

$$\boxed{\text{rank}(A) = 2}$$

which means that the spatial gradients provide independent information about both horizontal and vertical motion.

If

$$\text{rank}(A) = 1,$$

the local image structure does not provide enough independent information to determine both motion components reliably.

If

$$\text{rank}(A) = 0,$$

the spatial gradients contain essentially no useful information.

11 Image Preprocessing

The input video consists of color frames. For optical-flow estimation, the frames are converted into grayscale images.

The grayscale image can be represented as

$$I(x, y, t).$$

In the implementation,

```
cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

performs this conversion.

The grayscale image is then converted to floating-point representation:

```
astype(np.float64).
```

Floating-point representation is useful because gradient and temporal-difference calculations involve subtraction and can produce non-integer values.

12 Gaussian Smoothing

Before calculating image gradients, Gaussian smoothing is applied:

$$I_{\text{blur}} = G_{\sigma} * I,$$

where G_{σ} is a Gaussian kernel and $*$ denotes convolution.

The implementation uses

```
cv2.GaussianBlur(prev_gray, (5,5), 0)
```

and similarly for the current frame.

The purpose of Gaussian smoothing is to reduce high-frequency noise before computing derivatives.

This is useful because numerical differentiation can amplify noise.

13 Spatial Image Gradients

The spatial derivatives

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y}$$

describe how image intensity changes in the horizontal and vertical directions.

The implementation computes these derivatives using Sobel filters.

For the horizontal derivative,

$$I_x \approx \text{Sobel}_x(I).$$

For the vertical derivative,

$$I_y \approx \text{Sobel}_y(I).$$

The corresponding implementation is

```
Ixfinal = cv2.Sobel(prev, cv2.CV_64F, 1, 0, ksize=3)
Iyfinal = cv2.Sobel(prev, cv2.CV_64F, 0, 1, ksize=3)
```

Thus, the code obtains the spatial gradient information required by the optical-flow equation.

14 Temporal Gradient

The temporal derivative is approximated using the difference between the current and previous blurred frames:

$$I_t \approx I(x, y, t + \Delta t) - I(x, y, t).$$

The implementation uses

```
Itfinal = curr_blur - prev_blur
```

so that

$$I_t = I_{\text{current}} - I_{\text{previous}}.$$

The optical-flow equation is

$$I_x u + I_y v = -I_t.$$

Therefore, the right-hand side vector used in the linear system is

$$b = -I_t.$$

15 Construction of the Local Matrix

The image is divided into local windows.

For a window of size $w \times w$, the number of pixels in each window is

$$N = w^2.$$

For the implementation,

$$w = 20.$$

Therefore,

$$N = 20^2 = 400.$$

Hence,

$$A \in \mathbb{R}^{400 \times 2}.$$

Each pixel contributes one row:

$$\begin{bmatrix} I_x & I_y \end{bmatrix}.$$

Thus,

$$A = \begin{bmatrix} I_{x,1} & I_{y,1} \\ I_{x,2} & I_{y,2} \\ \vdots & \vdots \\ I_{x,400} & I_{y,400} \end{bmatrix}.$$

Similarly,

$$b = - \begin{bmatrix} I_{t,1} \\ I_{t,2} \\ \vdots \\ I_{t,400} \end{bmatrix}.$$

The optical-flow vector is then obtained by solving

$$A \begin{bmatrix} u \\ v \end{bmatrix} \approx b.$$

16 Why Multiple Pixels Are Used

A single pixel provides

$$I_x u + I_y v = -I_t,$$

which is one equation with two unknowns.

A local window provides many equations:

$$A\mathbf{v} = \mathbf{b}.$$

For a 20×20 window,

$$400$$

equations are used to estimate only two unknowns:

$$u, \quad v.$$

The redundancy makes the estimate more robust to noise and allows the unknown motion vector to be estimated through least squares.

The fundamental Lucas–Kanade assumption is that all pixels inside the local window have approximately the same optical flow.

17 Window Processing in the Implementation

The image dimensions are obtained as

$$H = \text{image height}, \quad W = \text{image width}.$$

The number of complete windows in the vertical and horizontal directions is calculated as

$$N_H = \left\lfloor \frac{H}{w} \right\rfloor,$$

and

$$N_W = \left\lfloor \frac{W}{w} \right\rfloor.$$

The implementation then processes every complete window using nested loops. For each window, the corresponding portions of

$$I_x, \quad I_y, \quad I_t$$

are extracted.

The pixels are then converted into the matrix A and vector $\mathbf{b}$.

18 Optical-Flow Vector

For every local window, the solution is

$$\mathbf{v} = \begin{bmatrix} u \\ v \end{bmatrix}.$$

The component u represents horizontal image motion:

$$u > 0 \quad \Rightarrow \quad \text{motion toward increasing } x.$$

Similarly,

$$u < 0 \quad \Rightarrow \quad \text{motion toward decreasing } x.$$

The component v represents vertical image motion:

$$v > 0 \quad \Rightarrow \quad \text{motion toward increasing } y,$$

while

$$v < 0 \quad \Rightarrow \quad \text{motion toward decreasing } y.$$

The optical-flow magnitude is

$$\|\mathbf{v}\| = \sqrt{u^2 + v^2}.$$

The direction is determined by

$$\theta = \tan^{-1} \left(\frac{v}{u} \right),$$

with the appropriate quadrant taken into account.

19 Visualization of Optical Flow

The estimated flow vectors are visualized using arrows.

For each window, the center of the window is calculated as

$$x_c = j w + \frac{w}{2},$$

$$y_c = i w + \frac{w}{2}.$$

The arrow starts at

$$(x_c, y_c).$$

The estimated flow vector is

$$(u, v).$$

For visualization, the vector is multiplied by a scale factor s :

$$x_{\text{end}} = x_c + su,$$

$$y_{\text{end}} = y_c + sv.$$

The implementation uses

$$s = 12.$$

Therefore,

$$(x_{\text{end}}, y_{\text{end}}) = (x_c + 12u, y_c + 12v).$$

The scale factor is used only to make the motion vectors visually noticeable. It does **not** modify the estimated optical flow itself.

20 Frame-to-Frame Processing

The video is processed sequentially.

First, the initial frame is read and converted to grayscale:

$$I_t.$$

Then the next frame is read:

$$I_{t+\Delta t}.$$

The spatial gradients and temporal gradient are calculated using the two frames.

The optical flow between these frames is then estimated.

After processing the current frame, it becomes the previous frame for the next iteration:

$$I_t \leftarrow I_{t+\Delta t}.$$

Thus, the process becomes

$$(I_1, I_2) \rightarrow (I_2, I_3) \rightarrow (I_3, I_4) \rightarrow \dots$$

This produces an optical-flow estimate between every pair of consecutive frames.

21 Output Video Generation

The processed frames are written into a new video using OpenCV's video writer.

The output video is created using

```
fourcc = cv2.VideoWriter_fourcc(*'XVID')

and

out = cv2.VideoWriter(
    'lucas_kanade_output.avi',
    fourcc,
    fps,
    (widths, heights)
)
```

For every processed frame,

```
out.write(curr_frame)
```

writes the frame into the output video.

Finally,

```
out.release()
```

is used to finalize and close the video file.

22 Complete Mathematical Pipeline

The complete Lucas–Kanade procedure implemented in the code can be summarized as follows.

Step 1: Read two consecutive frames

$$I(x, y, t), \quad I(x, y, t + \Delta t).$$

Step 2: Convert to grayscale

$$I_t^{(g)} = \text{Gray}(I_t).$$

Step 3: Apply Gaussian smoothing

$$\tilde{I}_t = G_\sigma * I_t^{(g)},$$

$$\tilde{I}_{t+\Delta t} = G_\sigma * I_{t+\Delta t}^{(g)}.$$

Step 4: Compute spatial gradients

$$I_x = \frac{\partial \tilde{I}_t}{\partial x},$$

$$I_y = \frac{\partial \tilde{I}_t}{\partial y}.$$

Step 5: Compute temporal gradient

$$I_t = \tilde{I}_{t+\Delta t} - \tilde{I}_t.$$

Step 6: Construct the local system

For each window,

$$A = \begin{bmatrix} I_{x,1} & I_{y,1} \\ \vdots & \vdots \\ I_{x,N} & I_{y,N} \end{bmatrix},$$

and

$$b = - \begin{bmatrix} I_{t,1} \\ \vdots \\ I_{t,N} \end{bmatrix}.$$

Step 7: Estimate optical flow

Solve

$$A \begin{bmatrix} u \\ v \end{bmatrix} \approx b.$$

Using least squares,

$$\begin{bmatrix} u \\ v \end{bmatrix} = \arg \min_{\mathbf{z}} \|A\mathbf{z} - b\|_2^2$$

and, under the full-column-rank condition,

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b.$$

Step 8: Visualize

Draw an arrow from the center of each window in the direction

$$(u, v).$$

Step 9: Store the processed frame

Write the frame into the output video.

23 Important Assumptions

The Lucas–Kanade method relies on several assumptions.

23.1 Brightness Constancy

The intensity of a moving point is assumed to remain approximately constant:

$$I(x, y, t) \approx I(x + dx, y + dy, t + dt).$$

23.2 Small Motion

The Taylor expansion assumes that

$$dx, dy, dt$$

are sufficiently small so that higher-order terms can be neglected.

23.3 Local Constant Motion

All pixels within a local window are assumed to have approximately the same optical-flow vector:

$$u_i \approx u, \quad v_i \approx v.$$

23.4 Sufficient Image Texture

Reliable estimation requires spatial intensity variation.

If both

$$I_x \approx 0$$

and

$$I_y \approx 0,$$

then the local image contains little information about motion.

24 Advantages

The Lucas–Kanade method has several advantages:

- It is conceptually simple.
- It has a clear mathematical derivation.
- It converts optical-flow estimation into a local least-squares problem.
- It can be implemented without using a built-in optical-flow function.
- Local averaging through the window makes the estimate more robust than using a single pixel.
- It is computationally manageable for short videos and moderate image sizes.

25 Limitations

The method also has several limitations.

25.1 Large Motion

The first-order Taylor approximation assumes small motion. Large displacement between consecutive frames can violate this assumption.

25.2 Brightness Changes

If illumination changes significantly, then

$$I(x, y, t) \neq I(x + dx, y + dy, t + dt),$$

and the brightness-constancy assumption becomes inaccurate.

25.3 Occlusion

When an object becomes occluded or newly visible, the same physical point may not be present in both frames.

25.4 Aperture Problem

Local image structures such as edges may not provide sufficient information to estimate both motion components.

25.5 Window Size

A very small window may not contain enough information.

A very large window may violate the assumption that all pixels have the same motion.

Therefore, the window size represents a trade-off between local detail and robustness.

26 Computational Interpretation

For every local window, the algorithm performs the following operations:

Image gradients $\rightarrow$ Local matrix $A \rightarrow$ Least-squares solution $\rightarrow (u, v)$
--

For a 20×20 window,

$$A \in \mathbb{R}^{400 \times 2}.$$

Therefore, the algorithm solves many small linear least-squares problems rather than one large global problem.

If the image contains many windows, the same procedure is repeated independently for each window.

The major computational cost comes from repeatedly solving these local least-squares systems for every window in every pair of consecutive frames.

27 Computation Versus Visualization

It is important to distinguish the actual optical-flow estimate from the way it is displayed.

The estimated flow is

$$(u, v).$$

The code then uses

$$s(u, v)$$

for drawing, where s is the visualization scale.

For example, if

$$(u, v) = (0.5, 0.2)$$

and

$$s = 12,$$

the arrow displacement becomes

$$(6, 2).$$

The underlying optical-flow estimate remains

$$(0.5, 0.2).$$

Therefore, changing the value of `scale` changes only the appearance of the arrows and does not change the Lucas–Kanade calculation.

28 Interpretation of the Final Result

The final output video contains the original video frames with estimated optical-flow vectors superimposed on them.

Each arrow represents the estimated local motion of the corresponding image region between the previous frame and the current frame.

The arrow direction indicates the direction of image motion, while the arrow length is proportional to the estimated flow magnitude after applying the visualization scale.

Thus, the output provides a visual representation of the motion field

$$\mathbf{V}(x, y) = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix}.$$

29 Summary

The Lucas–Kanade optical-flow method begins with the brightness-constancy assumption

$$I(x, y, t) = I(x + dx, y + dy, t + dt),$$

which leads, after first-order Taylor expansion, to the optical-flow constraint equation

$$I_x u + I_y v + I_t = 0.$$

Since one pixel provides only one equation for two unknowns, Lucas–Kanade considers a local window and assumes that all pixels in that window have approximately the same motion.

The resulting system is

$$A \begin{bmatrix} u \\ v \end{bmatrix} = b$$

with

$$A = \begin{bmatrix} I_{x,1} & I_{y,1} \\ \vdots & \vdots \\ I_{x,N} & I_{y,N} \end{bmatrix}, \quad b = - \begin{bmatrix} I_{t,1} \\ \vdots \\ I_{t,N} \end{bmatrix}.$$

The flow vector is obtained by solving the least-squares problem

$$\min_{u,v} \left\| A \begin{bmatrix} u \\ v \end{bmatrix} - b \right\|_2^2.$$

For a full-column-rank matrix,

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b.$$

The implementation performs this estimation independently for local windows across consecutive video frames and visualizes each estimated vector using an arrow.

Hence, the overall pipeline is

Video → Grayscale → Gaussian Smoothing → Spatial Gradients → Temporal Gradient → Local Leas